ACME Delivery Services

Online Analytics Pipeline

Data Engineering Take-Home Assessment - Solution Document

1. Executive Summary 2
2. Architecture Overview 2
   2.1 Component Stack 2
   2.2 Data Flow 2
   2.3 Spark Cluster Sizing Strategy 3
   2.4 Key Spark Metrics 3
   2.5 Docker Setup 3
3. Dimensional Data Model 4
   3.1 Design Decisions 4
   3.2 Schema 4
   3.3 SCD Type 4 Implementation 4
4. CDC and Streaming Design 5
   4.1 Debezium Envelope Parsing 5
   4.2 Schema Drift Handling with _additional_columns 5
   4.3 Upsert Pattern (foreachBatch + Staging + MERGE) 5
   4.4 Deduplication Strategy 6
5. Business Queries 6
6. FastAPI Backend 7
7. React Frontend 7
8. Running the Pipeline 7
   8.1 Prerequisites 7
   8.2 Start the Docker Stack 8
   8.3 Run the Spark Pipeline 8
   8.4 Run the FastAPI Backend 8
   8.5 Run the React Frontend 8
9. Known Limitations and Future Improvements 8

1. Executive Summary

This document describes the design and implementation of a near-real-time analytical pipeline for ACME Delivery Services. The pipeline ingests Change Data Capture (CDC) events from a PostgreSQL transactional database via Debezium and Kafka, processes them using Apache Spark Structured Streaming 3.5.0, and serves the results through a FastAPI backend and React frontend.

The solution addresses all three must-have requirements: historical and current order state queries, a dimensional data model, and near-real-time data availability.

2. Architecture Overview

2.1 Component Stack

Layer	Technology	Purpose
Source DB	PostgreSQL 14	Transactional database for orders, customers, products, order_items
CDC	Debezium (Kafka Connect)	Captures WAL changes and publishes to Kafka topics
Message Broker	Apache Kafka	Durable ordered event log for CDC events
Stream Processor	PySpark Structured Streaming 3.5.0	Consumes Kafka topics, parses Debezium envelope, upserts into analytical schema
Analytical Sink	PostgreSQL (analytical schema)	Dimensional model serving business queries
Backend API	FastAPI (Python)	Exposes 4 business query endpoints over the analytical schema
Frontend	React + Vite	Polls API every 10 seconds and displays near-real-time metrics
Orchestration	Docker Compose	Runs all services in a reproducible containerised environment

2.2 Data Flow

- PostgreSQL stored procedures generate and update orders, customers, and products on a pg_cron schedule.
- Debezium captures WAL changes and publishes CDC events to four Kafka topics.
- PySpark Structured Streaming reads from all four topics in parallel using foreachBatch micro-batches triggered every 10 seconds.
- Each micro-batch parses the Debezium envelope, deduplicates by primary key, writes to staging via JDBC, then executes a PostgreSQL INSERT ON CONFLICT MERGE into the target table.

- FastAPI queries the analytical schema views and returns JSON to the React frontend.
- The React frontend polls the API every 10 seconds to display near-real-time metrics.

2.3 Spark Cluster Sizing Strategy

The Spark job runs inside a Docker container based on apache/spark:3.5.0 in local[4] mode. This configuration was chosen to give each of the 4 concurrent streams its own thread, preventing JDBC write tasks from being blocked by the Kafka reader threads. This was the root cause of hanging JDBC writes observed during development on local[2].

Parameter	Value	Rationale
Master	local[4]	One thread per stream; prevents JDBC write deadlocks
Driver Memory	2g	Sufficient for micro-batch dedup window functions across 4 streams
Shuffle Partitions	2	Reduces overhead; default 200 causes unnecessary task spawning for small batches
Trigger Interval	10 seconds	Balances latency vs. PostgreSQL write pressure
Spark Version	3.5.0	Matches kafka package spark-sql-kafka-0-10_2.12:3.5.0

2.4 Key Spark Metrics

Metric	Location	What to Look For
inputRowsPerSecond	Spark UI - Streaming tab	Should match Kafka producer rate
processedRowsPerSecond	Spark UI - Streaming tab	Should be >= inputRowsPerSecond
batchDuration (ms)	Spark UI - Streaming tab	Should stay well under 10,000ms trigger interval
triggerExecution	Spark UI - Streaming tab	Time in foreachBatch; dominated by JDBC write and MERGE SQL
kafkaOffsetLag	Conduktor UI	Should trend toward 0; persistent lag means Spark cannot keep up

2.5 Docker Setup

The Spark consumer runs as a dedicated Docker service in the shared finance-shared-net Docker network alongside PostgreSQL, Kafka, and Debezium.

- Base image: apache/spark:3.5.0 with Python 3, psycopg2-binary, and pyspark==3.5.0 installed.

- Kafka and PostgreSQL hostnames are injected via environment variables (KAFKA_HOST, DB_HOST) so config.py resolves them correctly inside the Docker network.
- Checkpoints are persisted to a named Docker volume so Spark resumes from the correct Kafka offsets on container restart.
- Spark UI is exposed on port 4040 for monitoring micro-batch metrics during assessment review.

3. Dimensional Data Model

3.1 Design Decisions

- fact_orders holds the current state of each order. Primary table for Q1, Q2, and Q4.
- fact_order_items holds one row per order line. Used for Q3 (pending items by product).
- dim_customer and dim_product are slowly changing dimension tables holding the latest state.
- fact_orders_history, dim_customer_history, and dim_product_history hold the full CDC change history, implementing SCD Type 4.

3.2 Schema

Table	Type	Primary Key	Purpose
fact_orders	Fact (current state)	order_id	Latest state of each order. Drives Q1, Q2, Q4.
fact_orders_history	Fact (history)	history_id (BIGSERIAL)	Append-only. Every CDC event for orders.
fact_order_items	Fact	order_item_id	One row per order line. Drives Q3.
dim_customer	Dimension (current)	customer_id	Latest state of each customer.
dim_customer_history	Dimension (history)	history_id (BIGSERIAL)	Full CDC history for customers.
dim_product	Dimension (current)	product_id	Latest state of each product.
dim_product_history	Dimension (history)	history_id (BIGSERIAL)	Full CDC history for products.
*_staging	Staging (throwaway)	-	Spark writes micro-batch here; MERGE runs from here.

3.3 SCD Type 4 Implementation

SCD Type 4 separates current state from historical state into two tables. This satisfies the requirement to query both historical information and current order state with low latency.

- Current state table (e.g. fact_orders): upserted via INSERT ON CONFLICT DO UPDATE. Always reflects the latest known state.
- History table (e.g. fact_orders_history): plain INSERT on every CDC event, no ON CONFLICT. Every row is a point-in-time snapshot with a recorded_at timestamp.
- Full order lifecycle query: SELECT * FROM analytical.fact_orders_history WHERE order_id = X ORDER BY recorded_at ASC.

4. CDC and Streaming Design

4.1 Debezium Envelope Parsing

Every Debezium CDC event follows the same envelope structure: before, after, and op fields. The op field values are c (create), u (update), r (snapshot read), and d (delete). A shared utility function parse_debezium_envelope() accepts any table schema and dynamically wraps it into the full envelope structure, eliminating boilerplate across all 4 consumers.

4.2 Schema Drift Handling with _additional_columns

A critical design consideration was schema drift: what happens when the source database adds a column not yet in the Spark schema?

The solution is the _additional_columns field: a TEXT column in every target table that stores any fields present in the Debezium after payload but absent from the defined Spark schema, serialised as JSON.

- Expected fields are derived from the caller-supplied schema.
- The raw Kafka message is parsed by a UDF; any key in after not in the expected fields is serialised to JSON.
- Stored in _additional_columns alongside the known columns.

The pipeline never crashes on schema changes. New columns are captured automatically and extracted via PostgreSQL JSON operators until a formal migration is applied.

Real example encountered during development: the source order_items table had a typo quanity instead of quantity. Because quantity was not matched by the schema, the value landed in _additional_columns. The analytical views were updated to use COALESCE(oi.quantity, (oi._additional_columns::json->>'quanity')::INTEGER, 0) to handle both old and new data transparently.

4.3 Upsert Pattern (foreachBatch + Staging + MERGE)

Spark Structured Streaming does not natively support upserts into JDBC sinks. The pattern used throughout this pipeline is:

- Truncate staging before writing.
- Write the micro-batch DataFrame to staging via JDBC append mode.
- Execute INSERT ON CONFLICT DO UPDATE from staging into the target table via psycopg2.
- For history tables, execute a plain INSERT (no ON CONFLICT) from staging into the history table.
- Truncate staging after successful MERGE.

Staging tables use LIKE INCLUDING DEFAULTS EXCLUDING CONSTRAINTS so Spark can write without FK violations when child rows arrive before parent rows.

4.4 Deduplication Strategy

Within a single micro-batch, the same primary key can appear multiple times. Without deduplication, INSERT ON CONFLICT fails with: ON CONFLICT DO UPDATE command cannot affect row a second time.

- Spark-side: A Window function partitioned by primary key, ordered by updated_at DESC, keeps only the latest row per key before writing to staging.
- SQL-side: DISTINCT ON (primary_key) ORDER BY primary_key, updated_at DESC NULLS LAST in the MERGE SQL provides a safety net for any duplicates that slip through.

For history tables, deduplication is intentionally not applied; every CDC event is a valid point-in-time record.

5. Business Queries

Query	View	SQL Logic
Q1: Open orders by delivery date and status	v_open_orders_by_delivery	COUNT(order_id) GROUP BY delivery_date, status WHERE status != COMPLETED
Q2: Top 3 delivery dates with most open orders	v_top3_delivery_dates	COUNT(order_id) GROUP BY delivery_date WHERE status != COMPLETED ORDER BY count DESC LIMIT 3
Q3: Pending items by product	v_pending_items_by_product	SUM(quantity) JOIN fact_orders WHERE status = PENDING GROUP BY product_id
Q4: Top 3 customers with most pending orders	v_top3_pending_customers	COUNT(order_id) JOIN dim_customer WHERE status = PENDING GROUP BY customer_id LIMIT 3

Note on Q3: The query joins fact_order_items to fact_orders to read order status directly from the source of truth rather than relying on the denormalized order_status column in fact_order_items. This avoids potential sync drift.

6. FastAPI Backend

The serving layer is a FastAPI application exposing four endpoints, one per business query. Each endpoint queries the corresponding PostgreSQL view and returns a JSON response with a generated_at timestamp.

Endpoint	View Queried	Response Shape
/api/v1/order-delivery-summary	v_open_orders_by_delivery	{ data: [{delivery_date, status, order_count}], generated_at }
/api/v1/top-delivery-dates	v_top3_delivery_dates	{ data: [{delivery_date, open_order_count}], generated_at }
/api/v1/pending-items-by-product	v_pending_items_by_product	{ data: [{product_id, product_name, pending_quantity}], generated_at }
/api/v1/top-pending-customers	v_top3_pending_customers	{ data: [{customer_id, customer_name, pending_order_count}], generated_at }

Queries run against pre-built views so the API contains no raw SQL. Changing business logic means updating the view, not the API code.

7. React Frontend

The frontend is a React application built with Vite. It polls the FastAPI backend every 10 seconds using setInterval inside a useEffect hook, matching the Spark trigger interval.

- A useRef(true) flag tracks first load so the loading spinner only appears on initial render, not on subsequent polls, preventing the table from flickering every 10 seconds.
- Each API response is validated against a runtime contract. If the shape does not match, an error is displayed rather than rendering malformed data.
- Cleanup on unmount clears the interval to prevent memory leaks.

8. Running the Pipeline

8.1 Prerequisites

- Docker Desktop running
- Python 3.11+ with virtual environment activated

- Java 11+ with JAVA_HOME set
- HADOOP_HOME set with winutils.exe (Windows only)

8.2 Start the Docker Stack

```
docker compose up -d
```

8.3 Run the Spark Pipeline

Via Docker (recommended):

```
docker compose -f docker-compose.spark.yml up -d
```

Or locally:

```
spark-submit --master local[4] --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0,org.postgresql:postgresql:42.7.3 implementation/main.py
```

8.4 Run the FastAPI Backend

```
uvicorn api.main:app --reload --port 8001
```

8.5 Run the React Frontend

```
cd frontend && npm install && npm run dev
```

Frontend: http://localhost:5173 | Spark UI: http://localhost:4040 | Conduktor: http://localhost:8087

9. Known Limitations and Future Improvements

Area	Current State	Future Improvement
FK constraints	Removed from fact_order_items to handle CDC event ordering lag	Implement retry queue to resolve FK violations gracefully
Delete handling	op=d filtered out; soft deletes via is_active=false	Handle hard deletes by setting is_active=false on op=d events
Scale	local[4] single-node Spark in Docker	Deploy to Spark cluster scaled to Kafka partition count
Sink	PostgreSQL analytical schema	Replace with ClickHouse or Redshift for columnar analytics at scale
Frontend	Polling every 10 seconds	Add WebSocket support for push-based updates

Area	Current State	Future Improvement
Schema migration	_additional_columns JSON fallback	Implement Apicurio schema registry for formal schema evolution
Monitoring	Spark UI only	Add Prometheus and Grafana for persistent metrics and consumer lag alerting